

MÉMOIRE DE MASTER

Informatique — Intelligence Artificielle

Les

modèles de langage embarqués
sur smartphone Solutions Google (Gemini Nano, Flash, Lite)
et alternatives open source

ÉTAT

DE L'ART — VERSION COMPLÈTE

Thèmes couverts : Quantification · Distillation · LLaMA · Gemini
Nano/Flash/Lite
Phi-3 · MLC-LLM · llama.cpp · Benchmarks COTS · Edge+Cloud
Capacités comparées · Contraintes Android · Frameworks côte-à-côte

Année académique 2024–2025

Contents

Introduction	3
1 Fondements : des grands modèles aux modèles embarqués	3
1.1 L'émergence des capacités à grande échelle	3
1.2 La quantification : compresser sans dégrader	3
1.3 Fine-tuning efficace : QLoRA	4
1.4 Distillation de connaissance	4
2 Panorama des modèles disponibles pour smartphone	4
2.1 La famille LLaMA (Meta AI)	4
2.2 Phi-3 Mini (Microsoft Research)	4
2.3 La famille Gemini de Google : Nano, Flash et Lite	5
2.4 MobileLLM (Meta AI Research)	5
2.5 PowerInfer-2 : dépasser la contrainte RAM	6
2.6 Apple Intelligence	6
3 Comparaison des capacités qualitatives	6
3.1 Raisonnement et arithmétique	7
3.2 Instruction-following et utilité pratique	7
3.3 Support multilingue	7
3.4 Tâches pratiques sur mobile et contraintes Android	7
4 Frameworks d'inférence mobile	8
4.1 llama.cpp	8
4.2 MLC-LLM	8
4.3 ML Kit GenAI APIs	9
4.4 MediaPipe LLM Inference API	9
4.5 Portage et défis pratiques	9
5 Contraintes matérielles des smartphones	9
5.1 Architecture SoC : CPU, GPU et NPU	9
5.2 Mémoire vive et bande passante	10
5.3 Consommation énergétique et thermique	10
6 Études de performance sur appareils réels	10
6.1 Benchmark systématique sur appareils COTS	10
6.2 Analyse fine des phases d'inférence : lm-Meter	10

7	Exécution locale pure vs hybride (edge + cloud)	10
7.1	Exécution locale pure	11
7.2	Architecture hybride edge + cloud	11
7.3	Apple Private Cloud Compute : la référence hybride	11
8	Tendances et perspectives	12
8.1	Décodage spéculatif	12
8.2	Modèles multimodaux embarqués	12
8.3	Quantification mixte et adaptative	12
8.4	LLM as a System Service	12
	Conclusion	12
	Références bibliographiques	13

Introduction

L'essor des grands modèles de langage (Large Language Models, LLMs) depuis la publication de GPT-3 par Brown et al. en 2020 [1] a ouvert une problématique centrale : comment déployer ces systèmes aux capacités remarquables sur des dispositifs à ressources limitées ? Alors que GPT-3 mobilise 175 milliards de paramètres et nécessite plusieurs centaines de gigaoctets de mémoire GPU, la recherche s'est orientée vers des modèles compacts capables de s'exécuter directement sur smartphones, sans dépendance permanente au cloud. On désigne ce paradigme par l'expression **inférence embarquée** ou *on-device inference*.

Cette tendance répond à des enjeux concrets. Selon GSMA Intelligence [2], on compte en 2024 plus de 5,6 milliards d'abonnés mobiles uniques dans le monde (69 % de la population mondiale). Le smartphone constitue aujourd'hui le premier dispositif d'accès à l'information numérique, y compris dans des zones à connectivité intermittente.

Le présent état de l'art couvre : les fondements théoriques d'optimisation, le panorama complet des modèles (dont les variantes Gemini Nano/Flash/Lite), la comparaison des capacités qualitatives, les frameworks d'inférence, les performances mesurées sur appareils réels, et les architectures d'exécution locale vs. hybride edge+cloud.

1. Fondements : des grands modèles aux modèles embarqués

1.1 L'émergence des capacités à grande échelle

Brown et al. [1] démontrent avec GPT-3 que les capacités émergentes (few-shot, zero-shot) n'apparaissent pas progressivement mais brusquement à partir d'un certain seuil de paramètres. GPT-3 pose la question fondatrice : comment rendre ces capacités accessibles sur un appareil de 6 Go de RAM ?

1.2 La quantification : compresser sans dégrader

Dettmers et al. [7] introduisent **LLM.int8()**, qui résout le problème des *outliers* par une décomposition mixte (0,1 % en FP16, reste en INT8) : division de la mémoire par deux avec <1 % de dégradation.

Frantar et al. [8] franchissent une étape supplémentaire avec **GPTQ** : descente à INT4

par minimisation d'erreur couche par couche. LLaMA 7B passe de 14 Go (FP16) à ~ 4 Go (INT4).

Le format **GGUF** [9] standardise la distribution de ces modèles dans un fichier unique portable ($>100\,000$ fichiers sur HuggingFace en 2024).

INT4 comme sweet spot : Une synthèse récente [10] conclut que INT4 représente le meilleur compromis qualité/performance pour les modèles 3–7B sur flagship. Réduction documentée de 68,66 % pour LLaMA 3.2 3B après GPTQ INT4.

1.3 Fine-tuning efficace : QLoRA

Dettmers et al. [11] introduisent **QLoRA** : quantification NF4 + adaptateurs LoRA bas-rang. Fine-tuning d'un 65B sur un seul GPU A100 48 Go. Google l'exploite dans ML Kit GenAI [3] via des adaptateurs LoRA par tâche.

1.4 Distillation de connaissance

Hinton et al. [12] : un modèle étudiant imite les *soft targets* d'un enseignant. Omniprésent dans les modèles mobiles : LLaMA 3.2 1B/3B distillés depuis 8B, Apple Intelligence [13] depuis un MoE 64 experts (-90% coût d'entraînement).

2. Panorama des modèles disponibles pour smartphone

2.1 La famille LLaMA (Meta AI)

Les versions 1B et 3B de LLaMA 3.2 sont les seules réalistes pour smartphone (ARM64). En Q4_K_M : LLaMA 3.2 1B $\approx 0,8$ Go disque / 1,5 Go RAM ; LLaMA 3.2 3B ≈ 2 Go / 2,8 Go RAM.

2.2 Phi-3 Mini (Microsoft Research)

Abdin et al. [14] : Phi-3 Mini (3,8B) dépasse Mistral 7B sur MMLU (68,8 % vs 61,7 %) et atteint 82,5 % sur GSM8K grâce à des données synthétiques distillées depuis GPT-4. Licence MIT. Environ 3 Go RAM en Q4_K_M.

2.3 La famille Gemini de Google : Nano, Flash et Lite

Gemini Nano

Google DeepMind publie Gemini [15] en décembre 2023. Gemini Ultra atteint 90,0 % sur MMLU. La version **Nano** est déclinée en deux tailles :

- **Nano-1** (1,8B) : tâches de résumé et suggestion de réponse ; déployé sur Pixel 8 Pro via AICore.
- **Nano-2** (~3,25B) : 79,6 % sur MMLU, surpassant plusieurs modèles deux fois plus grands ; moteur de Pixel 9 et Galaxy S25.

Les deux variantes sont accessibles *uniquement* via AICore — service système Android gérant le cycle de vie du modèle, l'accélération NPU et l'isolation de confidentialité. L'accès développeur se fait via ML Kit GenAI APIs [3] ou MediaPipe LLM Inference API [17].

Gemini Flash et Flash-Lite

Gemini Flash est une famille de modèles légers conçue pour une inférence rapide dans des contextes *hybrides* (edge + cloud). La variante **Gemini 2.0 Flash** (2025) est accessible via l'API Gemini depuis une application mobile avec une très faible latence réseau. **Flash-Lite** cible les cas d'usage haute-fréquence où le coût et la vitesse priment sur la qualité maximale [16].

Distinction importante : Gemini Flash et Flash-Lite ne s'exécutent **pas** localement sur le smartphone. Ce sont des modèles cloud optimisés pour la faible latence. Ils s'intègrent dans une architecture *hybride* où l'application décide selon la complexité de la tâche de solliciter Gemini Nano on-device ou Gemini Flash via l'API.

Gemma (open source)

Gemma [6], version open source de Gemini sous licence commerciale permissive. Gemma 2 2B (distillé depuis 27B) rivalise avec GPT-3.5 selon les évaluateurs humains. Gemma 3 (2025) introduit des variantes 1B, 4B et 12B avec support multimodal natif. Testable via llama.cpp [4] ou MLC-LLM [5] sur tout Android ARM64.

2.4 MobileLLM (Meta AI Research)

Liu et al. [18] : à l'échelle sub-milliard, l'architecture prime sur les données. Un modèle profond et fin surpasse systématiquement un modèle large et plat de même taille.

MobileLLM 1B approche LLaMA 2 7B sur les tâches d’API mobiles avec $\sim 0,9$ Go de RAM.

2.5 PowerInfer-2 : dépasser la contrainte RAM

Xue et al. [19] exploitent la sparsité des activations ($\sim 30\%$ neurones actifs/token) en streamant le reste depuis le stockage flash. PowerInfer-2 exécute TurboSparse-Mixtral 47B sur smartphone à 11,68 tokens/s — $22\times$ vs llama.cpp.

2.6 Apple Intelligence

Apple ML Research [13] : modèle ~ 3 B compressé à 2 bits/poids (QAT). KV Cache Sharing : $-37,5\%$ mémoire et TTFT. Surpasse Phi-3 Mini, Mistral 7B, Gemma 7B et LLaMA 3 8B selon évaluateurs humains. $\sim 1,4$ Go RAM.

Table 1. Comparaison des principaux modèles pour déploiement mobile

Modèle	Éd.	Params	MMLU	RAM	Exécution
LLaMA 3.2 1B	Meta	1B	$\sim 49\%$	$\sim 1,5$ Go	On-device
LLaMA 3.2 3B	Meta	3B	61,8 %	$\sim 2,8$ Go	On-device
Gemma 2 2B	Google	2B	$\sim 52\%$	$\sim 2,0$ Go	On-device
Phi-3 Mini	MS	3,8B	68,8 %	$\sim 3,0$ Go	On-device (MIT)
Gemini Nano-1	Google	1,8B	—	$\sim 1,5$ Go*	On-device (AICore) *
Gemini Nano-2	Google	3,25B	79,6 %	~ 3 Go*	On-device (AICore)
Gemini Flash	Google	—	—	N/A	Hybride (API)
Gemini Fl.-Lite	Google	—	—	N/A	Hybride (API)
MobileLLM 1B	Meta	<1 B	$\sim 52\%$	$\sim 0,9$ Go	On-device
Apple Int. 3B	Apple	~ 3 B	$>$ Phi-3	$\sim 1,4$ Go**	On-device (iOS)

via AICore uniquement. ** quantification 2 bits QAT.

3. Comparaison des capacités qualitatives

Au-delà du seul score MMLU, les capacités pratiques divergent selon quatre axes : *raisonnement*, *instruction-following*, *support multilingue* et *tâches pratiques mobiles*.

3.1 Raisonnement et arithmétique

Le benchmark GSM8K (problèmes arithmétiques en langage naturel) est un indicateur robuste du raisonnement de base. Phi-3 Mini [14] domine avec 82,5 % — résultat remarquable pour 3,8B paramètres. LLaMA 3.2 3B atteint ~58 % ; Gemma 2 2B ~46 %. Les modèles sub-milliard (MobileLLM) montrent des limitations importantes sur les tâches multi-étapes (GSM8K <20 %). Le raisonnement logique reste le talon d'Achille général : aucun modèle <4B ne dépasse 70 % sur ARC-Challenge.

3.2 Instruction-following et utilité pratique

- **Gemini Nano-2** excelle sur les tâches de résumé, reformulation et complétion guidée grâce aux adaptateurs QLoRA par tâche [3].
- **Phi-3 Mini** présente le meilleur instruction-following général parmi les modèles open source <4B, grâce à ses données synthétiques orientées dialogue.
- **LLaMA 3.2 3B** offre de bonnes performances en conversation mais décroche sur les instructions complexes à contraintes multiples.
- **Gemma 2 2B** est particulièrement adapté à la classification et l'extraction d'entités, mais moins performant sur les réponses longues structurées.

3.3 Support multilingue

- **Gemini Nano** : support multilingue robuste natif, performances en français proches de l'anglais pour les tâches de résumé et complétion.
- **Gemma 2 2B** : couverture correcte pour les 35 langues du préentraînement Gemini.
- **Phi-3 Mini** : préentraînement orienté anglais ; performances en français dégradées (~15–20 % de baisse observée sur les tâches de QA).
- **LLaMA 3.2** : couverture limitée aux 8 langues officielles (dont le français) avec biais anglophone marqué.

3.4 Tâches pratiques sur mobile et contraintes Android

Quatre catégories de tâches : **résumé de contenu** (emails, articles), **suggestion de réponse** (clavier intelligent), **classification locale** (spam, sentiment), **extraction d'information** (NER, structuration).

Contraintes Android selon l'API level et le SoC :

- **ML Kit GenAI / AICore** : Android 10+, appareils certifiés uniquement (Pixel

9/10, Galaxy S25/S26 avec Tensor G4/G5 ou Snapdragon 8 Elite). Modèle pré-téléchargé par le système ($\sim 1,5\text{--}3\text{ Go}$), partagé entre applications.

- **llama.cpp via Termux** : compatible Android 7+ sur tout ARM64, sans restriction matérielle. Modèles téléchargés depuis HuggingFace par l'utilisateur.
- **MLC-LLM APK** : Android 8+, GPU Vulkan recommandé (OpenCL en fallback). Modèles compilés spécifiquement par cible.
- **MediaPipe LLM API [17]** : Android 10+, Java/Kotlin avec GPU delegate. Supporte Gemma, Phi-2, Falcon 1B en format LiteRT/TFLite.

Table 2. Comparaison des capacités qualitatives des modèles embarqués

Modèle	Raisonnement	Instruct.	Multilingual	Tâches mobiles
Gemini Nano-2	Moyen	Excellent	Excellent	Résumé, relecture
Phi-3 Mini	Excellent	Très bon	Moyen	QA, code
LLaMA 3.2 3B	Bon	Bon	Moyen	Chat, QA
Gemma 2 2B	Moyen	Bon	Bon	Classification, NER
MobileLLM 1B	Faible	Moyen	Faible	Suggestion, auto-complétion
Gemini Flash	Élevé	Excellent	Excellent	Toutes (hybride)

4. Frameworks d'inférence mobile

4.1 llama.cpp

Gerganov [4] : implémentation C/C++ zéro-dépendance, ARM NEON + AVX2. Format GGUF [9] natif. Installe dans Termux sans root. Limitation GPU Mali : accélération quasi inutilisable sur Samsung/MediaTek [21].

4.2 MLC-LLM

MLC AI [5] : compilateur TVM par cible matérielle. GPU Mali via Vulkan supporté. 20–25 % plus rapide que llama.cpp sur Snapdragon 8 Gen 3 et Dimensity 9300. APK Android directement installable. Format de modèles compilés propriétaire.

4.3 ML Kit GenAI APIs

Google [3] : APIs Kotlin/Java officielles sur AICore depuis 2025. Summarization, Proofreading, Rewriting, Image Description, prompts libres. Adaptateurs QLoRA par tâche. Limité aux flagships certifiés, quota d'inférence par application.

4.4 MediaPipe LLM Inference API

Google MediaPipe [17] offre une alternative plus flexible : supporte Gemma, Phi-2 et Falcon 1B via le format LiteRT (TFLite quantifié). Ne requiert pas d'appareil certifié — tout Android 10+ avec GPU décent peut faire tourner ces modèles. Pipeline unifié (tokenisation + inférence + décodage) accessible depuis Java, Kotlin et C++.

4.5 Portage et défis pratiques

Fassold [22] confirme un écart iOS/Android de $3\times$ (Apple Neural Engine vs GPU OpenCL) et un throttling thermique de 15–25 % sur milieu de gamme après 5 minutes.

Table 3. Comparaison des frameworks d'inférence mobile

Critère	llama.cpp	MLC-LLM	ML Kit GenAI	MediaPipe LLM
Modèles	GGUF universel	TVM compilé	Gemini Nano seul	Gemma, Phi-2, Falcon
GPU Mali	Non	Oui (Vulkan)	NPU AICore	Oui (Vulkan/OpenCL)
Android min.	Android 7+	Android 8+	Android 10+ cert.	Android 10+
Appareils	Tout ARM64	Tout (Vulkan rec.)	Flagships cert.	Tout (GPU)
API dév.	CLI/C++	Python/C++/Web	Kotlin/Java	Kotlin/Java/C++
Format	GGUF	MLC	AICore (syst.)	LiteRT/TFLite
Perf. relative	1×	~1,2–1,25×	~1,5–2×	~1,0–1,1×
Open source	Oui (MIT)	Oui (Apache 2)	Non	Oui (Apache 2)

5. Contraintes matérielles des smartphones

5.1 Architecture SoC : CPU, GPU et NPU

Le Snapdragon 8 Gen 3 [20] : CPU Cortex-X4 3,3 GHz, GPU Adreno 750 (4,7 TFLOPS FP16), NPU Hexagon (98 TOPS INT8), 16 Go LPDDR5X. Le NPU Hexagon accélère le prefill d'un facteur $50\times$ vs CPU sur LLaMA 2 7B [21].

5.2 Mémoire vive et bande passante

Android consomme 4–6 Go sur les 12–16 Go disponibles. Le decode est *memory-bound* : Snapdragon 8 Gen 3 offre ~ 77 GB/s (LPDDR5X) contre ~ 34 GB/s pour un Exynos 1380 (LPDDR4X), expliquant l'essentiel de leur écart de performances.

5.3 Consommation énergétique et thermique

5–10 % de batterie par 10 minutes de génération intensive [21]. Les NPUs réduisent cette consommation de 3–5 $\times$ vs GPU. Thermal throttling : 10–20 % de dégradation en session longue (15–25 % sur milieu de gamme [22]).

6. Études de performance sur appareils réels

6.1 Benchmark systématique sur appareils COTS

Xu et al. [21] couvrent 11 appareils réels avec llama.cpp et MLC-LLM. Snapdragon 8 Gen 3 : 30–56 tokens/s prefill (INT8). MLC-LLM : 20–25 % au-dessus de llama.cpp. Thermal throttling : 10–20 % en session longue.

Table 4. Performances comparatives sur appareils mobiles (modèles 7B Q4)

SoC	Appareil	Prefill	Decode	Throttling	Réf.
Snapdragon 8 Gen 3	Xiaomi 14 Pro	30–56 t/s	18–25 t/s	~ 10 –15 %	[21]
Dimensity 9300	Vivo X100	25–40 t/s	16–22 t/s	~ 10 –15 %	[21]
Apple M1	iPad	40–60 t/s	20–28 t/s	~ 10 %	[21]
Kirin 9000E	Huawei P60	15–20 t/s	10–14 t/s	~ 20 %	[21]
Exynos 1380	Galaxy A54	12–18 t/s	8–12 t/s	~ 15 –25 %	[22]

6.2 Analyse fine des phases d'inférence : lm-Meter

Xu et al. [23] : profileur de latence avec overhead < 3 %. Résultat clé : prefill = *compute-bound* (CPU/NPU), decode = *memory-bound* (bande passante RAM).

7. Exécution locale pure vs hybride (edge + cloud)

7.1 Exécution locale pure

Avantages : confidentialité totale, disponibilité hors-ligne, latence faible, coût nul. Limitations : modèles 1B–7B en pratique, qualité insuffisante pour le raisonnement multi-étapes complexe.

Yin et al. [24] proposent le LLM comme service OS mutualisé (−72 % RAM). Ye et al. [25] avec **prima.cpp** : inférence distribuée sur cluster Wi-Fi domestique, 674 ms/token pour un modèle 70B avec <6 % RAM par appareil.

7.2 Architecture hybride edge + cloud

L'approche hybride combine un modèle local léger pour les tâches simples avec un modèle cloud puissant pour les requêtes complexes. C'est le positionnement de **Gemini Flash** et **Flash-Lite** [16] : un routeur local décide selon la complexité de la requête si elle est traitée par Gemini Nano on-device ou transmise à Gemini Flash via l'API cloud.

Table 5. Comparaison des paradigmes d'exécution pour les LLMs mobiles

Critère	Local pur	Hybride	Cloud pur
Confidentialité	Totale	Partielle	Nulle
Qualité de réponse	Limitée (1–7B)	Élevée	Maximale
Disponibilité hors-ligne	Totale	Dégradée	Nulle
Consommation batterie	Élevée	Modérée	Faible
Latence	Faible	Variable	Réseau
Taille de modèle	1B–7B	Illimitée	Illimitée
Coût opérationnel	Nul	Modéré	Élevé

7.3 Apple Private Cloud Compute : la référence hybride

Apple [13] achemine vers Private Cloud Compute (PCC) les requêtes dépassant le modèle embarqué : serveurs Apple silicon avec garanties cryptographiquement vérifiables (données exclusivement pour la requête courante, supprimées immédiatement, inaccessibles même au personnel Apple).

8. Tendances et perspectives

8.1 Décodage spéculatif

Un modèle local génère des tokens candidats vérifiés par un modèle plus grand. Sur mobile, latence réduite de 2–3× dans les configurations hybrides. `Prima.cpp` [25] : 26 tokens/s pour un modèle 32B.

8.2 Modèles multimodaux embarqués

Gemini Nano [15] et Apple Intelligence [13] supportent déjà l'analyse d'images localement. Gemma 3 (2025) introduit le support vision natif pour les variantes 4B et 12B.

8.3 Quantification mixte et adaptative

Apple [13] avec 2 bits QAT (~1,4 Go pour 3B). QLoRA [11] reste la référence pour personnaliser les modèles compressés.

8.4 LLM as a System Service

Yin et al. [24] anticipent le LLM comme service OS, partiellement réalisé par AICore [3] et Apple Intelligence [13].

Conclusion

Le déploiement de LLMs sur smartphone est passé, en moins de deux ans, du stade expérimental à une réalité commerciale. Les avancées en quantification [7, 8], distillation [12] et architectures compactes [14, 18] ont rendu l'exécution locale viable. Les SoCs modernes [20] offrent une puissance suffisante pour des modèles de 1–4B [21, 22].

Deux trajectoires se dessinent : l'approche propriétaire (Google AICore + Gemini Nano [3], Apple Intelligence [13]) exploite les NPUs dédiés au prix d'une dépendance matérielle stricte ; l'écosystème open source (`llama.cpp` [4], MLC-LLM [5], Gemma [6]) offre une flexibilité maximale sur tout ARM64. Gemini Flash et Flash-Lite [16] couvrent le segment hybride à haute capacité.

Références bibliographiques

References

- [1] Brown, T., Mann, B., Ryder, N., et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS 33. [arXiv:2005.14165](https://arxiv.org/abs/2005.14165).
- [2] GSMA Intelligence (2024). *The Mobile Economy 2024*. GSMA, London.
- [3] Google Developers (2025). *ML Kit GenAI APIs Overview*. <https://developers.google.com/ml-kit/genai>.
- [4] Gerganov, G. (2023). *llama.cpp: LLM Inference in C/C++*. <https://github.com/ggml-org/llama.cpp>.
- [5] MLC AI Contributors (2023). *MLC-LLM: Bring Large Language Models Everywhere*. <https://github.com/mlc-ai/mlc-llm>.
- [6] Gemma Team, Google DeepMind (2024). *Gemma: Open Models Based on Gemini Research and Technology*. [arXiv:2403.08295](https://arxiv.org/abs/2403.08295).
- [7] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*. NeurIPS 2022. [arXiv:2208.07339](https://arxiv.org/abs/2208.07339).
- [8] Frantar, E., Ashkboos, S., Hoeffler, T., & Alistarh, D. (2022). *GPTQ: Accurate Post-Training Quantization for GPTs*. [arXiv:2210.17323](https://arxiv.org/abs/2210.17323).
- [9] llama.cpp Contributors (2023). *GGUF Format Specification*. <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>.
- [10] Anonymous Authors (2024). *Optimizing LLMs Using Quantization for Mobile Execution*. [arXiv:2512.06490](https://arxiv.org/abs/2512.06490).
- [11] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS 2023. [arXiv:2305.14314](https://arxiv.org/abs/2305.14314).
- [12] Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. NIPS Workshop. [arXiv:1503.02531](https://arxiv.org/abs/1503.02531).
- [13] Apple Machine Learning Research (2024). *Apple Intelligence Foundation Language Models*. [arXiv:2507.13575](https://arxiv.org/abs/2507.13575).
- [14] Abdin, M., et al. (2024). *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. Microsoft Research. [arXiv:2404.14219](https://arxiv.org/abs/2404.14219).

- [15] Google DeepMind (2023). *Gemini: A Family of Highly Capable Multimodal Models*. [arXiv:2312.11805](#).
- [16] Google DeepMind (2025). *Gemini 2.0 Flash and Flash-Lite: Fast, Efficient Models for Developers*. <https://ai.google.dev/gemini-api/docs/models>.
- [17] Google MediaPipe (2024). *LLM Inference Guide for Android — MediaPipe Solutions*. https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/android.
- [18] Liu, Z., Zhao, C., Iandola, F., et al. (2024). *MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases*. ICML 2024. [arXiv:2402.14905](#).
- [19] Xue, Z., Wei, Y., Chen, R., et al. (2024). *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. MobiCom 2024. ACM.
- [20] Qualcomm Technologies Inc. (2024). *Snapdragon 8 Gen 3 Mobile Platform. Technical Overview*. Qualcomm Developer Network.
- [21] Xu, D., et al. (2024). *Understanding LLMs in Your Pockets: Performance Study on COTS Mobile Devices*. [arXiv:2410.03613](#).
- [22] Fassold, H. (2024). *Porting LLMs to Mobile Devices for Question Answering*. IEEE/CVF CVPR Workshops 2024.
- [23] Xu, D., et al. (2024). *lm-Meter: Unveiling Runtime Inference Latency for On-Device Language Models*. [arXiv:2510.06126](#).
- [24] Yin, W., Xu, M., & Li, Y. (2024). *LLM as a System Service on Mobile Devices*. arXiv preprint.
- [25] Ye, Q., Li, Z., Feng, W., Guizani, M., & Yu, H. (2025). *Prima.cpp: Fast 30-70B LLM Inference on Heterogeneous Home Clusters*. [arXiv:2504.08791](#).